

计算机系统基础

程序的机器级表示(3)

王晶

jwang@ruc.edu.cn, 信息楼124

2025年10月



Data Manipulation

Arithmetic Operations

Instruction	Effect	Description
lea S, D	$D \leftarrow \&S$	Load effective address
inc D	$D \leftarrow D + 1$	Increment
dec D	$D \leftarrow D - 1$	Decrement
neg D	$D \leftarrow -D$	Negate
not D	$D \leftarrow \sim D$	Complement
add S, D	$D \leftarrow D + S$	Add
sub S, D	$D \leftarrow D - S$	Subtract
cmp S, D	$D - S$	Subtract
imull S, D	$D \leftarrow D * S$	Multiply



Arithmetic and Logical Operations

Logical Operations

Instruction	Effect	Description
xor S, D	$D \leftarrow D \wedge S$	Exclusive-or
or S, D	$D \leftarrow D \mid S$	Or
and S, D	$D \leftarrow D \& S$	And
test S, D	$D \& S$	And
sal k, D	$D \leftarrow D \ll k$	Left shift
shl k, D	$D \leftarrow D \ll k$	Left shift
sar k, D	$D \leftarrow D \gg k$	Arithmetic right shift
shr k, D	$D \leftarrow D \gg k$	Logical right shift



Special Arithmetic Operations

imull S	$R[\%edx]:R[\%eax] \leftarrow S * R[\%eax]$	Signed full multiply
mull S	$R[\%edx]:R[\%eax] \leftarrow S * R[\%eax]$	Unsigned full multiply
Cltld	$R[\%edx]:R[\%eax] \leftarrow \text{SignExtend}(R[\%eax])$	Convert to quad word
idiv S	$R[\%edx] \leftarrow R[\%edx]:R[\%eax] \bmod S$ $R[\%eax] \leftarrow R[\%edx]:R[\%eax] \div S$	Signed divide
divl S	$R[\%edx] \leftarrow R[\%edx]:R[\%eax] \bmod S$ $R[\%eax] \leftarrow R[\%edx]:R[\%eax] \div S$	Unsigned divide

Initially x at %ebp+8, y at %ebp+12

```
1 movl 8(%ebp), %eax
2 imull 12(%ebp)
3 pushl %edx
4 pushl %eax
```

```
1 movl 8(%ebp), %eax
2 cltd
3 idivl 12(%ebp)
4 pushl %eax
5 pushl %edx
```

Long arith2(long x, long y, long z)
{
Long t1 = [填空1]
Long t2 = [填空2]
Long t3 = [填空3]
Long t4 = [填空4]
Return t4;
}

- arith2:
 - #x@%rdi, y@rsi, z@rdx
 - Orq %rsi, %rdi
 - Sarq \$3, %rdi
 - Notq %rdi
 - Movq %rdx, %rax
 - Subq %rdi, %rax
 - Ret

作答



课堂练习

Long arith2(long x, long y,
long z) {

Long t1 = x | y;

Long t2 = t1 >> 3;

Long t3 = ~t2;

Long t4 = z-t3;

Return t4;

}

- Arith2:

- #x@%rdi, y@rsi, z@rdx

- orq %rsi, %rdi

- sarq \$3, %rdi

- notq %rdi

- movq %rdx, %rax

- subq %rdi, %rax

- ret

- 填写空缺的C语言语句



程序的控制

- 条件代码
- 条件分支
- 循环
- switch 语句



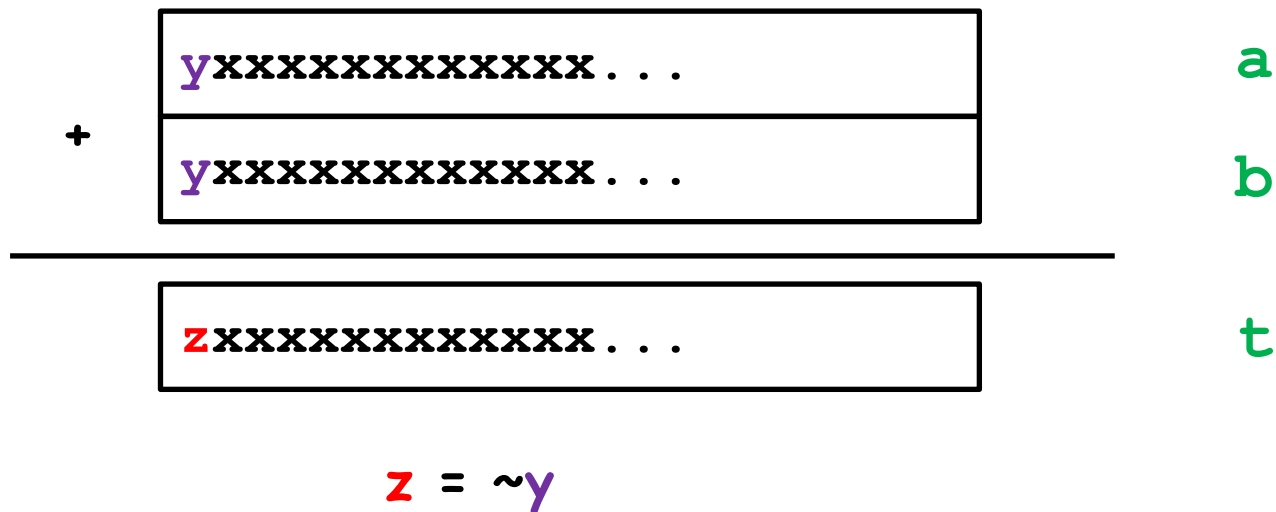
CF set when

	1xxxxxxxxxxxxxxxxx... + 1xxxxxxxxxxxxxxxxx...	进位
1	xxxxxxxxxxxxxxxxxx...	
1	0xxxxxxxxxxxxxxxxx... - 1xxxxxxxxxxxxxxxxx...	借位
	1xxxxxxxxxxxxxxxxx...	

For unsigned arithmetic, this reports overflow



OF set when



$(a > 0 \ \&\& \ b > 0 \ \&\& \ t < 0) \ || \ (a < 0 \ \&\& \ b < 0 \ \&\& \ t \geq 0)$

For signed arithmetic, this reports overflow



进位和溢出的区别

- 进位标志反映无符号整数运算结果是否超出范围
 - 有进位，加上进位或借位后运算结果仍然正确
- 溢出标志反映有符号整数运算结果是否超出范围
 - 有溢出，运算结果已经不正确
- 处理器按照无符号整数求得结果
 - 设置进位标志CF
 - 设置溢出标志OF
- 程序员决定
 - 操作数是无符号数，关心进位
 - 操作数是有符号数，注意溢出



零标志ZF (Zero Flag)

- 运算结果为0，则ZF=1，否则ZF=0

结果是0，
ZF标志不是0！

举例

- 8位二进制数相加：

$$00111010 + 01111100 = 10110110$$

结果不是0，ZF=0

- 8位二进制数相加：

$$10000100 + 01111100 = [1] \underline{00000000}$$

结果是0，ZF=1

进位

结果



符号标志SF (Sign Flag)

- 运算结果最高位为1，则SF=1；否则SF=0

最高位 = 符号位 = SF

举例

- 8位二进制数相加：

$$00111010 + 01111100 = 10110110$$

最高位 = 1: SF = 1

- 8位二进制数相加：

$$10000100 + 01111100 = [1]00000000$$

最高位 = 0: SF = 0

进位

结果



读取标志位

有符号数

无符号数

Instruction	Synonym	Effect	Set Condition
Sete	Setz	ZF	Equal/zero
Setne	Setnz	$\sim ZF$	Not equal/not zero
Sets		SF	Negative
Setns		$\sim SF$	Nonnegative
Setl	Setnge	$SF \wedge OF$	Less
Setle	Setng	$(SF \wedge OF) ZF$	Less or Equal
Setg	Setnle	$\sim (SF \wedge OF) \& \sim ZF$	Greater
Setge	Setnl	$\sim (SF \wedge OF)$	Greater or Equal
Seta	Setnbe	$\sim CF \& \sim ZF$	Above
Setae	Setnb	$\sim CF$	Above or equal
Setb	Setnae	CF	Below
Setbe	Setna	$CF ZF$	Below or equal



标志位用于比较

				flags				interpretation	
				CF	OF	SF	ZF	of xor sf signed	cf unsigned
1	3B	3B	00	0	0	0	1	op1=op2	op1=op2
2	3B	15	26	0	0	0	0	op1>op2	op1>op2
3	15	3B	DA	1	0	1	0	op1<op2	op1<op2
4	F9	F6	03	0	0	0	0	op1>op2	op1>op2
5	F6	F9	FD	1	0	1	0	op1<op2	op1<op2
6	15	F6	1F	1	0	0	0	op1>op2	op1<op2
7	F6	15	E1	0	0	1	0	op1<op2	op1>op2
8	68	A5	C3	1	1	1	0	op1>op2	op1<op2
9	A5	68	3D	0	1	0	0	op1<op2	op1>op2



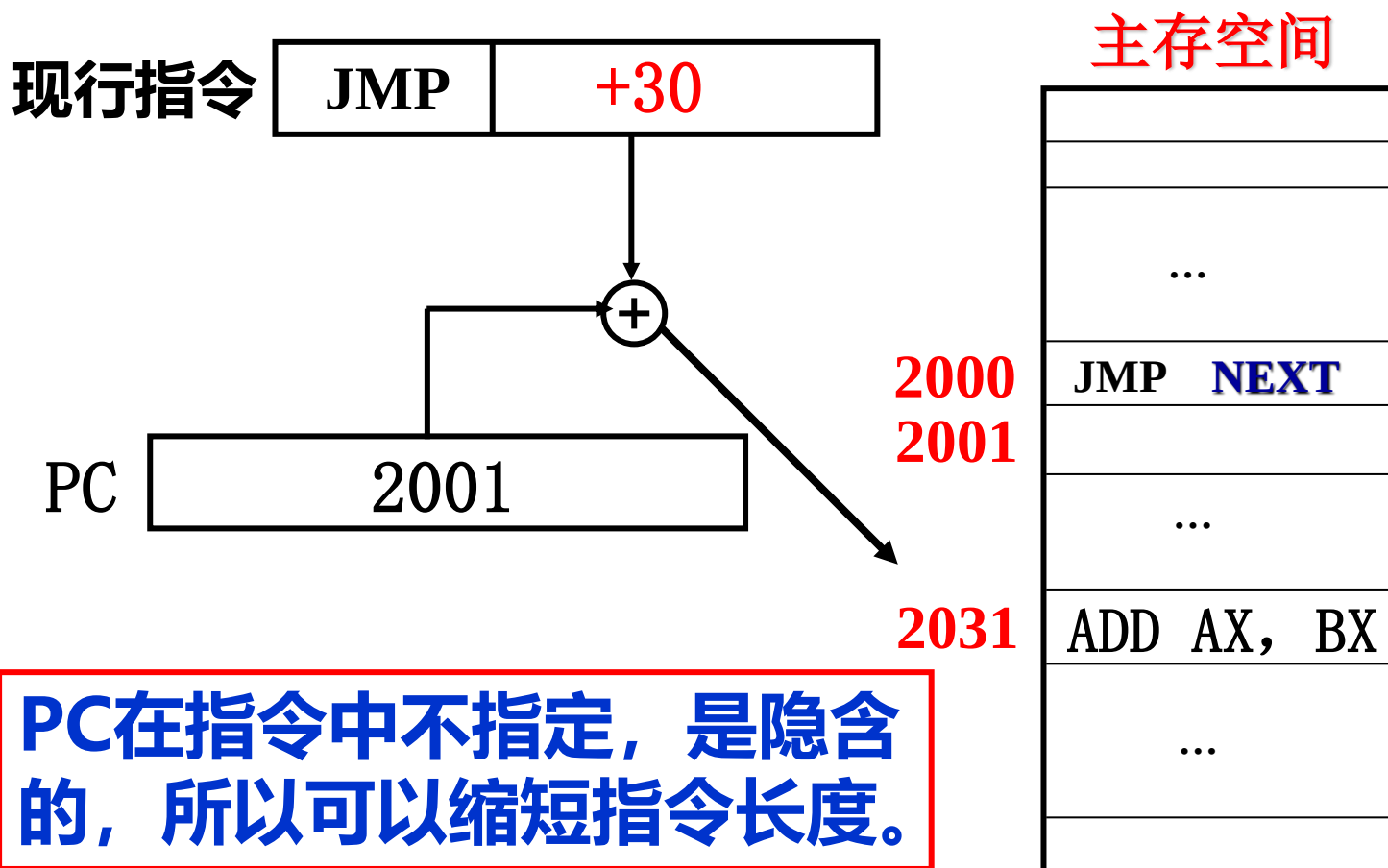
Today

- 标志位
- 条件跳转
- 循环
- Switch



相对寻址

- 指令的地址由程序计数器PC的内容和指令地址码相加得到
- 主要用于转移指令，缩短指令长度。





课堂练习

- 下列XXXX的值是什么？

- 1)

- 0x804828f: 74 05

je XXXX

- 0x8048291: e8 1e 00 00 00

call 80482b4

- 2)

- 0x8048357: 72 e7

jb XXXX

- 0x8048359: c6 05 10 a0 04 08 01

movb \$0x1, 0x804a010



- 1)

- 0x804828f: 74 05 je XXXX (0x8048296)
- 0x8048291: e8 1e 00 00 00 call 80482b4

- 2)

- 0x8048357: 72 e7 jb XXXX (0x8048340)
- 0x8048359: c6 05 10 a0 04 08 01 movb \$0x1, 0x804a010



课堂练习

• 3)

- XXXX: 74 12
- XXXX: b8 00 00 00 00

je 8048391
mov \$0x0, %eax

• 4)

- 80482bf: e9 e0 ff ff ff
- 80482c4: 90

jmp XXXX
nop



练习答案

• 3)

• 0x804837d: 74 12

je 8048391

• 0x804837f: b8 00 00 00 00

mov \$0x0, %eax

• 4)

• 80482bf: e9 e0 ff ff ff

jmp 0x80482a4

• 80482c4: 90

nop



条件表达式翻译-分支

C Code

```
val = Test ? Then_Expr : Else_Expr;
```

```
val = x > y ? x - y : y - x;
```

Goto Version

```
n_test = !Test;  
if (n_test) goto Else;  
val = Then_Expr;  
goto Done;  
Else:  
    val = Else_Expr;  
Done:  
    . . .
```

- Create separate code regions for then & else expressions
- Execute appropriate one



Conditional Move Example

```
long absdiff
(long x, long y)
{
    long result;
    if (x > y)
        result = x-y;
    else
        result = y-x;
    return result;
}
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value

absdiff:

```
movq    %rdi, %rax    # x
subq    %rsi, %rax    # result = x-y
movq    %rsi, %rdx
subq    %rdi, %rdx    # eval = y-x
cmpq    %rsi, %rdi    # x:y
cmovle  %rdx, %rax    # if <=, result = eval
ret
```



Bad Cases for Conditional Move

Expensive Computations

```
val = Test(x) ? Hard1(x) : Hard2(x);
```

Bad Performance

- 需要实现对两个分支情况都求值
- 只有操作都足够简单，才能抵消两次计算的开销

Risky Computations

```
val = p ? *p : 0;
```

Unsafe

- 某个分支语句有“副作用”（会修改某个寄存器或变量的值）时，不能使用 **Conditional Move**

Computations with side effects

```
val = x > 0 ? x*=7 : x+=3;
```

Illegal

- Both values get computed
- Must be side-effect free



Today

- Control: Condition codes
- Conditional branches
- Loops
- Switch Statements



General “Do-While” Translation

C Code

```
do  
    Body  
while (Test) ;
```

Goto Version

```
loop:  
    Body  
    if (Test)  
        goto loop
```

- Body: {
 Statement₁;
 Statement₂;
 ...
 Statement_n;
}



“Do-While” Loop Compilation

Goto Version

```
long pcount_goto
(unsigned long x) {
    long result = 0;
loop:
    result += x & 0x1;
    x >>= 1;
    if(x) goto loop;
    return result;
}
```

Register	Use(s)
%rdi	Argument x
%rax	result

```
        movl    $0, %eax    # result = 0
.L2:                                # loop:
        movq    %rdi, %rdx
        andl    $1, %edx    # t = x & 0x1
        addq    %rdx, %rax  # result += t
        shrq    %rdi        # x >>= 1
        jne     .L2         # if (x) goto loop
        rep
        ret
```



General “While” Translation #1

- “Jump-to-middle” translation
- Used with `-Og`

While version

```
while (Test)  
    Body
```



Goto Version

```
    goto test;  
loop:  
    Body  
test:  
    if (Test)  
        goto loop;  
done:
```



While Loop Example #1

C Code

```
long pcount_while
(unsigned long x) {
    long result = 0;
    while (x) {
        result += x & 0x1;
        x >>= 1;
    }
    return result;
}
```

Jump to Middle

```
long pcount_goto_jtm
(unsigned long x) {
    long result = 0;
    goto test;
loop:
    result += x & 0x1;
    x >>= 1;
test:
    if(x) goto loop;
    return result;
}
```

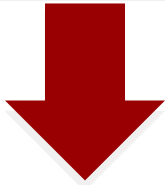
- Compare to do-while version of function
- Initial goto starts loop at test



General “While” Translation #2

While version

```
while (Test)  
  Body
```



Do-While Version

```
if (!Test)  
  goto done;  
do  
  Body  
  while (Test) ;  
done:
```



Goto Version

```
if (!Test)  
  goto done;  
loop:  
  Body  
  if (Test)  
    goto loop;  
done:
```

- “Do-while” conversion
- Used with -O1



While Loop Example #2

C Code

```
long pcount_while
(unsigned long x) {
    long result = 0;
    while (x) {
        result += x & 0x1;
        x >>= 1;
    }
    return result;
}
```

Do-While Version

```
long pcount_goto_dw
(unsigned long x) {
    long result = 0;
    if (!x) goto done;
    loop:
        result += x & 0x1;
        x >>= 1;
        if(x) goto loop;
    done:
        return result;
}
```

- Initial conditional guards entrance to loop
- Compare to do-while version of function
 - Removes jump to middle.



多判断条件的短路性

- 逻辑与运算

```
while (sum < 1000) and (count ≤ 24) loop  
    ... { body of loop }  
end while;
```

两个条件必须都成立 $\Leftrightarrow$ 任一个不成立都退出

```
whileSum:  cmpw $1000, $sum    ; sum < 1000 ?  
           jnl  endWhileSum    ; exit if not  
           cmpw $24, %cx       ; count ≤ 24 ?  
           jnle endWhileSum    ; exit if not  
           ...                 ; body of loop . .  
           jmp  whileSum       ; go check condition again  
  
endWhileSum:
```



多判断条件的短路性

- 逻辑或运算

```
while (sum < 1000) or (flag=1) loop
... { body of loop }
end while;
```

```
whileSum:    cmpw $1000, $sum    ; sum < 1000 ?
             jl  body           ; execute body if so
             cmpb $1, %dh       ; flag=1 ?
             jne endWhileSum    ; exit if not
body:        ...                ; body of loop . .
             ....
             jmp  whileSum      ; go check condition again
endWhileSum:
```




“For” Loop Form

General Form

```
for (Init; Test; Update )  
    Body
```

```
#define WSIZE 8*sizeof(int)  
long pcount_for  
    (unsigned long x)  
{  
    size_t i;  
    long result = 0;  
    for (i = 0; i < WSIZE; i++)  
    {  
        unsigned bit =  
            (x >> i) & 0x1;  
        result += bit;  
    }  
    return result;  
}
```

Init

```
i = 0
```

Test

```
i < WSIZE
```

Update

```
i++
```

Body

```
{  
    unsigned bit =  
        (x >> i) & 0x1;  
    result += bit;  
}
```



“For” Loop → While Loop

For Version

```
for (Init; Test; Update )  
    Body
```



While Version

```
Init;  
while (Test) {  
    Body  
    Update;  
}
```



For-While Conversion

Init

```
i = 0
```

Test

```
i < WSIZE
```

Update

```
i++
```

Body

```
{  
    unsigned bit =  
        (x >> i) & 0x1;  
    result += bit;  
}
```

```
long pcount_for_while  
(unsigned long x)  
{  
    size_t i;  
    long result = 0;  
    i = 0;  
    while (i < WSIZE)  
    {  
        unsigned bit =  
            (x >> i) & 0x1;  
        result += bit;  
        i++;  
    }  
    return result;  
}
```



“For” Loop Do-While Conversion

C Code

Goto Version

```
long pcount_for
(unsigned long x)
{
    size_t i;
    long result = 0;
    for (i = 0; i < WSIZE; i++)
    {
        unsigned bit =
            (x >> i) & 0x1;
        result += bit;
    }
    return result;
}
```

- Initial test can be optimized away – why?

```
long pcount_for_goto_dw
(unsigned long x) {
    size_t i;
    long result = 0;
    i = 0; Init
    if (!(i < WSIZE)) ! Test
    goto done;
loop:
{
    unsigned bit =
        (x >> i) & 0x1; Body
    result += bit;
}
i++; Update
if (i < WSIZE) Test
    goto loop;
done:
    return result;
}
```



Loop 指令实现 for 循环

- LOOP label

- 利用 ECX 计数器自动减1，方便实现计数循环的程序结构。
- $ECX \leftarrow ECX - 1$;dec ecx
- 如果 $ECX \neq 0$ ，转标号 label 继续循环 ;jnz label
- 否则继续执行循环指令下面的语句
- 操作数 label 采用相对短转移寻址方式，及向后128个字节或向前127个字节。



使用 “loop” 的80x86汇编代码

```
for count := 20 downto 1 loop
    ... { body of loop }
end for
```

forCount:	movl \$20, %ecx	; number of iterations
	...	; body of loop
	...	
	loop forCount	; repeat body 20 times

forIndex:	movl \$number, %ecx	; number of iterations
	...	; body of loop
	...	
	loop forIndex	; repeat body number times



可靠的 for 循环

- 当ecx中的值为0时，循环不会执行

```
    movl $number, %ecx    ; number of iterations
    cmpl $0, %ecx         ; number = 0 ?
    jle endFor            ; skip loop if number = 0
forIndex: ...             ; body of loop
    ...
    loop forIndex         ; repeat body number times
endFor:
```

```
    movl $number, %ecx    ; number of iterations
    jecxz endFor          ; skip loop if number = 0
forIndex: ...             ; body of loop
    ...
    loop forIndex         ; repeat body number times
endFor:
```



Today

- **Control: Condition codes**
- **Conditional branches**
- **Loops**
- **Switch Statements**



Switch 结构

- Switch结构特性
 - 整数 测试
 - 多结果 (可能是很多的分支情况)
 - 如果采用if/elseif分支，分支很多，需要做多次判断，产生计算开销（cmp）
 - 一次性处理所有分支，效率更高！
 - 改善代码可读性
- 标准
 - Case的数量，以及case值的分布
 - Case数量太少，还是用if-else方便
 - Case值分布太广、太稀疏，jump table开销过大



Switch 语句

```
int switch_eg(int x, int n)
{
    int result = x ;
    switch ( n ) {
        case 100:
            result *= 13 ;
            break ;
        case 102:
            result += 10 ;
            /* fall through */
```

*Integer
testing*

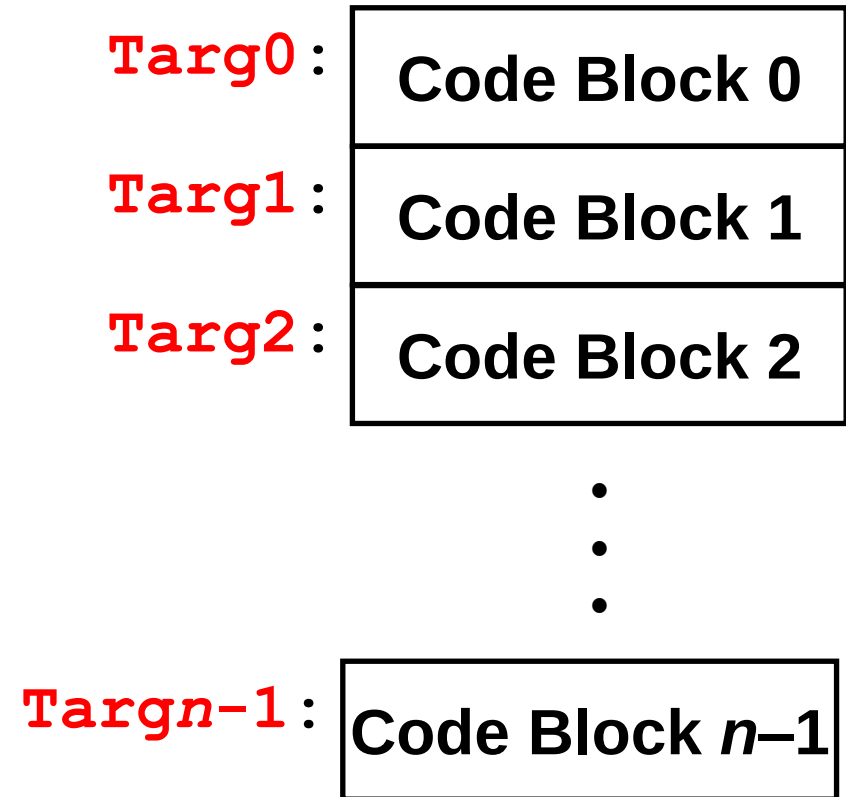
```
        case 103
            result += 11;
            break ;
        case 104: case 106:
            result *= result ;
            break ;
        default:
            result = 0 ;
    }
    return result ;
}
```

*Multiple
cases*



从 Cases 到 Targets

```
switch(op) {  
  case val_0:  
    Block 0  
  case val_1:  
    Block 1  
    ...  
  case val_n-1:  
    Block n-1  
}
```





构建跳转表

Jump Table

jtab:	Targ0	val_0
	:	
	Targ1	val_1
	:	
	Targ2	val_2
	:	
	Targ $n-1$	val_ $n-1$

Jump Targets

Targ0:	Code Block 0
Targ1:	Code Block 1
Targ2:	Code Block 2
	:
	:
	:
Targ $n-1$:	Code Block $n-1$



完成Switch

Approx. Translation

```
target = JTab[op];  
goto *target;
```

Jump Table

jtab:	Targ0	val_0
		
	Targ1	val_1
		
	Targ2	val_2
	⋮	
	Targ $n-1$	val_ $n-1$

Jump Targets

Targ0: Code Block 0

Targ1: Code Block 1

Targ2: Code Block 2

⋮

Targ $n-1$: Code Block $n-1$



Switch 举例

- Multiple case labels
 - Here: 5 & 6
- Fall through cases
 - Here: 2
- Missing cases
 - Here: 4

```
long switch_eg
(long x, long y, long z)
{
    long w = 1;
    switch(x) {
        case 1:
            w = y*z;
            break;
        case 2:
            w = y/z;
            /* Fall Through */
        case 3:
            w += z;
            break;
        case 5:
        case 6:
            w -= z;
            break;
        default:
            w = 2;
    }
    return w;
}
```



跳转表的结构

Switch Form

```
switch(x) {  
  case val_0:  
    Block 0  
  case val_1:  
    Block 1  
    . . .  
  case val_n-1:  
    Block n-1  
}
```

Translation (Extended C)

```
goto *JTab[x];
```

Jump Table

jtab:	Targ0
	Targ1
	Targ2
	•
	•
	•
	Targn-1

Jump Targets

Targ0:

Code Block
0

Targ1:

Code Block
1

Targ2:

Code Block
2

•
•
•

Targn-1:

Code Block
n-1



Switch 举例

```
long switch_eg(long x, long y, long z)
{
    long w = 1;
    switch(x) {
        . . .
    }
    return w;
}
```

Setup:

switch_eg:

```
movq    %rdx, %rcx
cmpq    $6, %rdi    # x:6
ja      .L8
jmp      ← *.L4(, %rdi, 8)
```

What range of values takes default?

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rdx	Argument z
%rax	Return value

Note that w not initialized here



Switch 举例

```
long switch_eg(long x, long y, long z)
{
    long w = 1;
    switch(x) {
        . . .
    }
    return w;
}
```

Setup:

```
switch_eg:
    movq    %rdx, %rcx
    cmpq    $6, %rdi        # x:6
    ja      .L8              # Use default
    jmp     *.L4(, %rdi, 8)   # goto *JTab[x]
```

Jump table

```
.section    .rodata
    .align  8
.L4:
    .quad    .L8 # x = 0
    .quad    .L3 # x = 1
    .quad    .L5 # x = 2
    .quad    .L9 # x = 3
    .quad    .L8 # x = 4
    .quad    .L7 # x = 5
    .quad    .L7 # x = 6
```

**Indirect
jump**





Assembly Setup Explanation

- 表结构

- 每个表项 8 bytes
- 首地址在 **.L4**

- 跳转

- **直接:** `jmp .L8`
- 跳转目标由 **.L8**表示

- **间接:** `jmp *.L4(, %rdi, 8)`

- 从表起始地址 **.L4**
- 每8字节一跳 (地址是64bit= 8 bytes)
- 取目标的有效地址 **.L4 + x*8**
 - 仅当 $0 \leq x \leq 6$

Jump table

```
.section      .rodata
    .align 8
.L4:
    .quad     .L8 # x = 0
    .quad     .L3 # x = 1
    .quad     .L5 # x = 2
    .quad     .L9 # x = 3
    .quad     .L8 # x = 4
    .quad     .L7 # x = 5
    .quad     .L7 # x = 6
```



跳转表

Jump table

```
.section      .rodata
.align 8
.L4:
.quad        .L8 # x = 0
.quad        .L3 # x = 1
.quad        .L5 # x = 2
.quad        .L9 # x = 3
.quad        .L8 # x = 4
.quad        .L7 # x = 5
.quad        .L7 # x = 6
```

```
switch(x) {
case 1:      // .L3
    w = y*z;
    break;
case 2:      // .L5
    w = y/z;
    /* Fall Through */
case 3:      // .L9
    w += z;
    break;
case 5:
case 6:      // .L7
    w -= z;
    break;
default:     // .L8
    w = 2;
}
```



代码块($x == 1$)

```
switch(x) {  
  case 1:      // .L3  
    w = y*z;  
    break;  
  . . .  
}
```

```
.L3:  
  movq    %rsi, %rax  # y  
  imulq   %rdx, %rax  # y*z  
  ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rdx	Argument z
%rax	Return value





处理直落情况

```
long w = 1;  
.  
.  
.  
switch(x) {  
.  
.  
.  
case 2:  
    w = y/z;  
    /* Fall Through */  
case 3:  
    w += z;  
    break;  
.  
.  
.  
}
```

case 2:
 w = y/z;
 goto merge;

case 3: w = 1;
merge: w += z;



代码块($x == 2, x == 3$)

```
long w = 1;
. . .
switch(x) {
. . .
case 2:
    w = y/z;
    /* Fall Through */
case 3:
    w += z;
    break;
. . .
}
```

```
.L5:                                # Case 2
    movq    %rsi, %rax
    cqto
    idivq   %rcx                    # y/z
    jmp     .L6                    # goto merge
.L9:                                # Case 3
    movl    $1, %eax               # w = 1
.L6:                                # merge:
    addq    %rcx, %rax             # w += z
    ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rdx	Argument z
%rax	Return value



代码块

(x == 5, x == 6, default)

```
switch(x) {  
    . . .  
    case 5:  // .L7  
    case 6:  // .L7  
        w -= z;  
        break;  
    default: // .L8  
        w = 2;  
}
```

```
.L7:                                # Case 5,6  
    movl    $1, %eax                # w = 1  
    subq    %rdx, %rax              # w -= z  
    ret  
.L8:                                # Default:  
    movl    $2, %eax                # 2  
    ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rdx	Argument z
%rax	Return value



课堂练习

填写C语言代码:

```
Long test (long x, long y) {  
    long val = 8x;  
    if (y>0) {  
        if (x<y) {  
            val = y-x;  
        } else if (y<=-2)  
            val = x+y;  
        return val;  
    }  
}
```

x @ %rdi, y @ %rsi

```
leaq    0(%rdi,8), %rax  
testq   %rsi, %rsi  
jle     .L2  
movq    %rsi, %rax  
subq    %rdi, %rax  
movq    %rdi, %rdx  
andq    %rsi, %rdx  
cmpq    %rsi, %rdi  
cmovge  %rdx, %rax  
.L2:  
addq    %rsi, %rdi  
cmpq    $-2, %rsi  
cmovle  %rdi, %rax  
ret
```




课堂练习

程序填空，变量映射关系，
并解释函数功能

```
Int fun_a(unsigned x) {  
    int val = _____  
    while ( _____ ) {  
        _____  
        _____  
    }  
    return _____;  
}
```

- `x@$ebp+8`
`movl 8(%ebp), %edx`
`movl $0, %eax`
`testl %edx, %edx`
`Je .L7`
`.L10:`
`xorl %edx, %eax`
`shr $1,%edx ;逻辑右移1位`
`jne .L10`
`.L7:`
`andl $1, %eax`



课堂练习

- 函数 `fun_b(unsigned long x)` {
 - `long val = 0;`
 - `long i;`
 - `for (... ; ... ; ...)` {
 - ...
 - }
 - `return val;`
- }
- Gcc生成的汇编代码如右所示。
- 完成下面工作：
 - A. 根据汇编代码，填写C代码缺失的部分；
 - B. 解释循环前为什么没有初始测试，也没有初始跳转到循环内部的测试部分；
 - C. 用自然语言描述这个函数的功能

```
# x in %rdi
1 func_b:
2     movl $64, %rdx
3     movl $0, %rax
4 .L10:
5     movq %rdi, %rcx
6     andl $1, %rcx
7     addq %rax, %rax
8     orq %rcx, %rax
9     shr $1, %rdi
10    sub $1, %rdx
11    jne .L10
12    ret
```



练习答案

- fun_b(unsigned long x) {
 - long val = 0;
 - long i;
 - for (i=64; i!=0 ; i--) {
 - Val = (val << 1) | (x&1)
 - X >>= 1;
 - }
 - return val;
- }
- 2) 因为循环次数是常量
- 3) 将x镜像反转

```
# x in %rdi
1 func_b:
2     movl $64, %rdx
3     movl $0, %rax
4     .L10:
5     movq %rdi, %rcx
6     andl $1, %rcx
7     addq %rax, %rax
8     orq %rcx, %rax
9     shr $1, %rdi
10    sub $1, %rdx
11    jne .L10
12    ret
```



练习

```
long switcher(long a, long b,
long c, long *dest)
{
    long val;
    switch(a) {
        case ____ :
            c = ____;
            /* Fall Through */
        case ____ :
            val = ____;
            break;
        case ____ :
        case ____ :
            val = ____;
            break;
        case ____ :
            val = ____;
            break;
        default:
            val = ____;
    }

    *dest=val
}
```

Switcher:

```
cmpq    $7, %rdi
ja      .L2
jmp     *.L4(,%rdi,8)
.section          .rodata
```

```
.L7:
xorq    $15, %ris
movq    %rsi, %rdi
.L3:
leaq    112(%rdx), %rdi
jmp     .L6
.L5:
leaq    (%rdx, %rsi), %rdi
salq    $2, %rdi
jmp     .L6
.L2:
movq     %rsi, %rdi
.L6:
movq     %rsi, (%rcx)
ret
```

```
.L4:
.quad    .L3
.quad    .L2
.quad    .L5
.quad    .L2
.quad    .L6
.quad    .L7
.quad    .L2
.quad    .L5
```



总结

- C Control
 - if-then-else
 - do-while
 - while, for
 - switch
- Assembler Control
 - Conditional jump
 - Conditional move
 - Indirect jump (via jump tables)
 - Compiler generates code sequence to implement more complex control
- Standard Techniques
 - Loops converted to do-while or jump-to-middle form
 - Large switch statements use jump tables
 - Sparse switch statements may use decision trees (if-elseif-elseif-else)



Finding Jump Table in Binary

```
00000000004005e0 <switch_eg>:
4005e0:    48 89 d1                mov     %rdx,%rcx
4005e3:    48 83 ff 06             cmp     $0x6,%rdi
4005e7:    77 2b                   ja      400614 <switch_eg+0x34>
4005e9:    ff 24 fd f0 07 40 00    jmpq    *0x4007f0(,%rdi,8)
4005f0:    48 89 f0                mov     %rsi,%rax
4005f3:    48 0f af c2             imul    %rdx,%rax
4005f7:    c3                      retq
4005f8:    48 89 f0                mov     %rsi,%rax
4005fb:    48 99                   cqto
4005fd:    48 f7 f9                idiv    %rcx
400600:    eb 05                   jmp     400607 <switch_eg+0x27>
400602:    b8 01 00 00 00          mov     $0x1,%eax
400607:    48 01 c8                add     %rcx,%rax
40060a:    c3                      retq
40060b:    b8 01 00 00 00          mov     $0x1,%eax
400610:    48 29 d0                sub     %rdx,%rax
400613:    c3                      retq
400614:    b8 02 00 00 00          mov     $0x2,%eax
400619:    c3                      retq
```



Finding Jump Table in Binary

```
00000000004005e0 <switch_eg>:
. . .
4005e9:      ff 24 fd f0 07 40 00      jmpq    *0x4007f0(,%rdi,8)
. . .
```

```
% gdb switch
(gdb) x /8xg 0x4007f0
0x4007f0:      0x0000000000400614      0x00000000004005f0
0x400800:      0x00000000004005f8      0x0000000000400602
0x400810:      0x0000000000400614      0x000000000040060b
0x400820:      0x000000000040060b      0x2c646c25203d2078
(gdb)
```



Finding Jump Table in Binary

```
% gdb switch
(gdb) x /8xg 0x4007f0
0x4007f0:      0x000000000000400614      0x0000000000004005f0
0x400800:      0x0000000000004005f8      0x000000000000400602
0x400810:      0x000000000000400614      0x00000000000040060b
0x400820:      0x00000000000040060b      0x2c646c25203d2078
```

...		
4005f0:	48 89 f0	mov %rsi,%rax
4005f3:	48 0f af c2	imul %rdx,%rax
4005f7:	c3	retq
4005f8:	48 89 f0	mov %rsi,%rax
4005fb:	48 99	cqto
4005fd:	48 f7 f9	idiv %rcx
400600:	eb 05	jmp 400607 <switch_eg+0x27>
400602:	b8 01 00 00 00	mov \$0x1,%eax
400607:	48 01 c8	add %rcx,%rax
40060a:	c3	retq
40060b:	b8 01 00 00 00	mov \$0x1,%eax
400610:	48 29 d0	sub %rdx,%rax
400613:	c3	retq
400614:	b8 02 00 00 00	mov \$0x2,%eax
400619:	c3	retq